

PL/SQL:

Exercise 1: Control Structures

```
1 DECLARE
2   -- Customer variables
3   TYPE customer_rec IS RECORD (
4     CustomerID NUMBER,
5     Name VARCHAR2(50),
6     Age NUMBER,
7     Balance NUMBER,
8     IsVIP VARCHAR2(5)
9   );
10
11   TYPE customer_tab IS TABLE OF customer_rec INDEX BY PLS_INTEGER;
12   customers customer_tab;
13
14   -- Loan variables
15   TYPE loan_rec IS RECORD (
16     CustomerID NUMBER,
17     InterestRate NUMBER,
18     DueInDays NUMBER
19   );
20
21   TYPE loan_tab IS TABLE OF loan_rec INDEX BY PLS_INTEGER;
22   loans loan_tab;
23
24 BEGIN
25   -- Sample Customer Data
26   customers(1).CustomerID := 1;
27   customers(1).Name := 'Rahul';
28   customers(1).Age := 65;
29   customers(1).Balance := 12000;
30   customers(1).IsVIP := 'FALSE';
31
32   customers(2).CustomerID := 2;
33   customers(2).Name := 'Priya';
34   customers(2).Age := 45;
35   customers(2).Balance := 8000;
36
37   customers(3).CustomerID := 3;
38   customers(3).Name := 'Amit';
39   customers(3).Age := 70;
40   customers(3).Balance := 15000;
41   customers(3).IsVIP := 'FALSE';
42
43   -- Loan Data
44   loans(1).CustomerID := 1;
45   loans(1).InterestRate := 10;
46   loans(1).DueInDays := 20;
47
48   loans(2).CustomerID := 2;
49   loans(2).InterestRate := 9;
50   loans(2).DueInDays := 45;
51
52   loans(3).CustomerID := 3;
53   loans(3).InterestRate := 11;
54   loans(3).DueInDays := 10;
55
56   DBMS_OUTPUT.PUT_LINE('----- Scenario 1 -----');
57
58   FOR i IN 1..customers.COUNT LOOP
59     IF customers(i).Age > 60 THEN
60       FOR j IN 1..loans.COUNT LOOP
61         IF loans(j).CustomerID = customers(i).CustomerID THEN
62           loans(j).InterestRate := loans(j).InterestRate - 1;
63           DBMS_OUTPUT.PUT_LINE(customers(i).Name ||
64             ' New Interest Rate = ' || loans(j).InterestRate || '%');
65         END IF;
66       END LOOP;
67     END IF;
68   END LOOP;
69
70   DBMS_OUTPUT.PUT_LINE(' ');
71   DBMS_OUTPUT.PUT_LINE('----- Scenario 2 -----');
72
73   FOR i IN 1..customers.COUNT LOOP
74     IF customers(i).Balance > 10000 THEN
75       customers(i).IsVIP := 'TRUE';
76       DBMS_OUTPUT.PUT_LINE(customers(i).Name || ' promoted to VIP');
77     END IF;
78   END LOOP;
79
80   DBMS_OUTPUT.PUT_LINE(' ');
81   DBMS_OUTPUT.PUT_LINE('----- Scenario 3 -----');
82
83   FOR i IN 1..customers.COUNT LOOP
84     FOR j IN 1..loans.COUNT LOOP
85       IF customers(i).CustomerID = loans(j).CustomerID
86         AND loans(j).DueInDays <= 30 THEN
87         DBMS_OUTPUT.PUT_LINE('Reminder: Dear ' ||
88           customers(i).Name ||
89           ', your loan is due in ' ||
90           loans(j).DueInDays || ' days.');
91       END IF;
92     END LOOP;
93   END LOOP;
94 END;
```

STON

Input for the program (optional)

Output:

```
----- Scenario 1 -----
Rahul New Interest Rate = 9%
Amit New Interest Rate = 10%
----- Scenario 2 -----
Rahul promoted to VIP
Amit promoted to VIP
----- Scenario 3 -----
Reminder: Dear Rahul, your loan is due in 20 days.
Reminder: Dear Amit, your loan is due in 10 days.
```

Light 13 Wiki Oracle PL/SQL

Exercise 2: Error Handling

```
1 DECLARE
2   -- Scenario 1 Variables
3   fromBalance NUMBER := 5000;
4   toBalance NUMBER := 3000;
5   transferAmount NUMBER := 6000;
6
7   -- Scenario 2 Variables
8   employeeID NUMBER := 105;
9   validEmployee NUMBER := 101;
10  salary NUMBER := 50000;
11  increasePercent NUMBER := 10;
12
13  -- Scenario 3 Variables
14  newCustomerID NUMBER := 101;
15  existingCustomerID NUMBER := 101;
16
17 BEGIN
18   DBMS_OUTPUT.PUT_LINE('==== Scenario 1: Safe Fund Transfer =====');
19
20   BEGIN
21     IF transferAmount > fromBalance THEN
22       RAISE_APPLICATION_ERROR(-20001,'Insufficient Funds');
23     END IF;
24
25     fromBalance := fromBalance - transferAmount;
26     toBalance := toBalance + transferAmount;
27
28     DBMS_OUTPUT.PUT_LINE('Transfer Successful');
29
30   EXCEPTION
31     WHEN OTHERS THEN
32       DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
33       DBMS_OUTPUT.PUT_LINE('Transaction Rolled Back');
34   END;
```

STDIN
Input for the program (Optional)

Output:
==== Scenario 1: Safe Fund Transfer =====
Error: ORA-20001: Insufficient Funds
Transaction Rolled Back
==== Scenario 2: Update Salary =====
Error: Employee ID does not exist.
==== Scenario 3: Add New Customer =====
Error: Customer ID already exists.

```
END;

DBMS_OUTPUT.PUT_LINE(' ');

DBMS_OUTPUT.PUT_LINE('==== Scenario 2: Update Salary =====');

BEGIN
  IF employeeID <> validEmployee THEN
    RAISE NO_DATA_FOUND;
  END IF;

  salary := salary + (salary * increasePercent / 100);
  DBMS_OUTPUT.PUT_LINE('Updated Salary: ' || salary);

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Error: Employee ID does not exist. ');
END;

DBMS_OUTPUT.PUT_LINE(' ');

DBMS_OUTPUT.PUT_LINE('==== Scenario 3: Add New Customer =====');

BEGIN
  IF newCustomerID = existingCustomerID THEN
    RAISE DUP_VAL_ON_INDEX;
  END IF;

  DBMS_OUTPUT.PUT_LINE('Customer Added Successfully. ');

EXCEPTION
  WHEN DUP_VAL_ON_INDEX THEN
    DBMS_OUTPUT.PUT_LINE('Error: Customer ID already exists. ');
```

Input for the program (Optional)

Output:
==== Scenario 1: Safe Fund Transfer =====
Error: ORA-20001: Insufficient Funds
Transaction Rolled Back
==== Scenario 2: Update Salary =====
Error: Employee ID does not exist.
==== Scenario 3: Add New Customer =====
Error: Customer ID already exists.

Exercise 3: Stored Procedures

<pre>1 DECLARE 2 3 -- Scenario 1: Process Monthly Interest 4 PROCEDURE ProcessMonthlyInterest IS 5 balance NUMBER := 10000; 6 interestRate NUMBER := 1; 7 BEGIN 8 balance := balance + (balance * interestRate / 100); 9 10 DBMS_OUTPUT.PUT_LINE('===== Scenario 1 ====='); 11 DBMS_OUTPUT.PUT_LINE('Interest Applied: 1%'); 12 DBMS_OUTPUT.PUT_LINE('Updated Balance: ' balance); 13 END; 14 15 -- Scenario 2: Update Employee Bonus 16 PROCEDURE UpdateEmployeeBonus(17 dept VARCHAR2, 18 bonusPercent NUMBER 19) IS 20 salary NUMBER := 50000; 21 BEGIN 22 salary := salary + (salary * bonusPercent / 100); 23 24 DBMS_OUTPUT.PUT_LINE(CHR(10) '===== Scenario 2 ====='); 25 DBMS_OUTPUT.PUT_LINE('Department: ' dept); 26 DBMS_OUTPUT.PUT_LINE('Bonus Percentage: ' bonusPercent '%'); 27 DBMS_OUTPUT.PUT_LINE('Updated Salary: ' salary); 28 END; 29 30 -- Scenario 3: Transfer Funds 31 PROCEDURE TransferFunds(32 fromBalance NUMBER, 33 toBalance NUMBER, 34 amount NUMBER 35) IS</pre>	<pre>STBIN Input for the program (optional) Output: ===== Scenario 1 ===== Interest Applied: 1% Updated Balance: 10100 ===== Scenario 2 ===== Department: IT Bonus Percentage: 10% Updated Salary: 55000 ===== Scenario 3 ===== Transfer Successful Sender Balance: 7000 Receiver Balance: 8000</pre>
<pre>-- Scenario 3: Transfer Funds PROCEDURE TransferFunds(fromBalance NUMBER, toBalance NUMBER, amount NUMBER) IS sender NUMBER := fromBalance; receiver NUMBER := toBalance; BEGIN DBMS_OUTPUT.PUT_LINE(CHR(10) '===== Scenario 3 ====='); IF sender >= amount THEN sender := sender - amount; receiver := receiver + amount; DBMS_OUTPUT.PUT_LINE('Transfer Successful'); DBMS_OUTPUT.PUT_LINE('Sender Balance: ' sender); DBMS_OUTPUT.PUT_LINE('Receiver Balance: ' receiver); ELSE DBMS_OUTPUT.PUT_LINE('Transfer Failed: Insufficient Balance'); END IF; END; BEGIN -- Calling all procedures ProcessMonthlyInterest; UpdateEmployeeBonus('IT',10); TransferFunds(10000,5000,3000); END; /</pre>	<pre>===== Input for the program (optional) Output: ===== Scenario 1 ===== Interest Applied: 1% Updated Balance: 10100 ===== Scenario 2 ===== Department: IT Bonus Percentage: 10% Updated Salary: 55000 ===== Scenario 3 ===== Transfer Successful Sender Balance: 7000 Receiver Balance: 8000</pre>

Exercise 4: Functions

```
1 DECLARE
2   dob DATE := TO_DATE('15-08-2000','DD-MM-YYYY');
3   age NUMBER;
4
5   loanAmount NUMBER := 500000;
6   interestRate NUMBER := 10;
7   years NUMBER := 5;
8   installment NUMBER;
9
10  balance NUMBER := 10000;
11  amount NUMBER := 5000;
12
13 BEGIN
14
15   -- Scenario 1: Calculate Age
16   age := TRUNC((SYSDATE - dob)/365);
17
18   DBMS_OUTPUT.PUT_LINE('==== Scenario 1 =====');
19   DBMS_OUTPUT.PUT_LINE('Customer Age = ' || age);
20
21   -- Scenario 2: Monthly Installment
22   installment := (loanAmount + (loanAmount*interestRate*years/100))/(years*12);
23
24   DBMS_OUTPUT.PUT_LINE(' ');
25   DBMS_OUTPUT.PUT_LINE('==== Scenario 2 =====');
26   DBMS_OUTPUT.PUT_LINE('Monthly Installment = ' || ROUND(installment,2));
27
28   -- Scenario 3: Sufficient Balance
29   DBMS_OUTPUT.PUT_LINE(' ');
30   DBMS_OUTPUT.PUT_LINE('==== Scenario 3 =====');
31
32   IF balance >= amount THEN
33     DBMS_OUTPUT.PUT_LINE('Customer has sufficient balance');
34   ELSE
35     DBMS_OUTPUT.PUT_LINE('Customer has insufficient balance');
```

STDIN
Input for the program (Optional)

Output:
==== Scenario 1 =====
Customer Age = 25
==== Scenario 2 =====
Monthly Installment = 12500
==== Scenario 3 =====
Customer has sufficient balance

Exercise 5: Triggers

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

```
1  /* =====
2  Exercise 5 : Triggers
3  ===== */
4
5  /* Scenario 1 */
6  CREATE OR REPLACE TRIGGER UpdateCustomerLastModified
7  BEFORE UPDATE ON Customers
8  FOR EACH ROW
9  BEGIN
10     :NEW.LastModified := SYSDATE;
11 END;
12 /
13
14 /* Scenario 2 */
15 CREATE OR REPLACE TRIGGER LogTransaction
16 AFTER INSERT ON Transactions
17 FOR EACH ROW
18 BEGIN
19     INSERT INTO AuditLog
20     VALUES(
21         :NEW.TransactionID,
22         :NEW.AccountID,
23         SYSDATE
24     );
25 END;
26 /
27
28 /* Scenario 3 */
29 CREATE OR REPLACE TRIGGER CheckTransactionRules
30 BEFORE INSERT ON Transactions
31 FOR EACH ROW
32 BEGIN
33
34     IF :NEW.TransactionType = 'Withdrawal'
35     AND :NEW.Amount > :NEW.Balance THEN
```

Input for the program (optional)

Output

BEFORE UPDATE ON Customers

*

ERROR at line 2:

ORA-00942: table or view "sandbox_user"."CUSTOMERS" does not exist

AFTER INSERT ON Transactions

*

ERROR at line 2:

ORA-00942: table or view "sandbox_user"."TRANSACTIONS" does not exist

BEFORE INSERT ON Transactions

*

ERROR at line 2:

ORA-00942: table or view "sandbox_user"."TRANSACTIONS" does not exist

Ready

Light 13 Wiki Oracle PL/SQL

Exercise 6: Cursors

DECLARE

```
CURSOR GenerateMonthlyStatements IS
  SELECT 'Rahul' CustomerName, 5000 Amount FROM DUAL
  UNION ALL
  SELECT 'Priya', 3000 FROM DUAL;
```

```
CURSOR ApplyAnnualFee IS
  SELECT 'A101' AccountNo, 10000 Balance FROM DUAL
  UNION ALL
  SELECT 'A102', 15000 FROM DUAL;
```

```
CURSOR UpdateLoanInterestRates IS
  SELECT 'L101' LoanID, 10 InterestRate FROM DUAL
  UNION ALL
  SELECT 'L102', 12 FROM DUAL;
```

BEGIN

```
DBMS_OUTPUT.PUT_LINE('==== Scenario 1 ====');
```

```
FOR rec IN GenerateMonthlyStatements LOOP
  DBMS_OUTPUT.PUT_LINE('Customer: ' || rec.CustomerName ||
    ' Transaction: ' || rec.Amount);
END LOOP;
```

```
DBMS_OUTPUT.PUT_LINE(' ');
```

```
DBMS_OUTPUT.PUT_LINE('==== Scenario 2 ====');
```

```
FOR acc IN ApplyAnnualFee LOOP
  DBMS_OUTPUT.PUT_LINE('Account: ' || acc.AccountNo);
  DBMS_OUTPUT.PUT_LINE('Balance Before: ' || acc.Balance);
  DBMS_OUTPUT.PUT_LINE('Balance After: ' || (acc.Balance - 500));
END LOOP;
```

STDIN

Input for the program (Optional)

Output:

```
==== Scenario 1 ====
Customer: Rahul Transaction: 5000
Customer: Priya Transaction: 3000
==== Scenario 2 ====
Account: A101
Balance Before: 10000
Balance After: 9500
Account: A102
Balance Before: 15000
Balance After: 14500
==== Scenario 3 ====
Loan ID: L101
Old Interest Rate: 10%
New Interest Rate: 9%
Loan ID: L102
Old Interest Rate: 12%
New Interest Rate: 11%
```

Exercise 7: Packages

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35

```
SET SERVEROUTPUT ON;

-----
-- PACKAGE 1 : CustomerManagement
-----

CREATE OR REPLACE PACKAGE CustomerManagement AS
    PROCEDURE AddCustomer;
    PROCEDURE UpdateCustomer;
    FUNCTION GetCustomerBalance RETURN NUMBER;
END CustomerManagement;
/

CREATE OR REPLACE PACKAGE BODY CustomerManagement AS
    PROCEDURE AddCustomer IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Customer Added Successfully');
    END;

    PROCEDURE UpdateCustomer IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Customer Details Updated');
    END;

    FUNCTION GetCustomerBalance RETURN NUMBER IS
    BEGIN
        RETURN 10000;
    END;
END CustomerManagement;
/

-----
-- PACKAGE 2 : EmployeeManagement
-----

CREATE OR REPLACE PACKAGE EmployeeManagement AS
    PROCEDURE HireEmployee;
    PROCEDURE UpdateEmployee;
    FUNCTION CalculateAnnualSalary RETURN NUMBER;
END EmployeeManagement;
/

CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS

    PROCEDURE HireEmployee IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Employee Hired Successfully');
    END;

    PROCEDURE UpdateEmployee IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Employee Updated Successfully');
    END;

    FUNCTION CalculateAnnualSalary RETURN NUMBER IS
    BEGIN
        RETURN 600000;
    END;

END EmployeeManagement;
/

-----
-- PACKAGE 3 : AccountOperations
-----

CREATE OR REPLACE PACKAGE AccountOperations AS
    PROCEDURE OpenAccount;
    PROCEDURE CloseAccount;
    FUNCTION GetTotalBalance RETURN NUMBER;
END AccountOperations;
/

CREATE OR REPLACE PACKAGE BODY AccountOperations AS

    PROCEDURE OpenAccount IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Account Opened Successfully');
    END;

    PROCEDURE CloseAccount IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Account Closed Successfully');
    END;

    FUNCTION GetTotalBalance RETURN NUMBER IS
    BEGIN
        RETURN 25000;
    END;

END AccountOperations;
/

-----
-- CALLING ALL PACKAGES
-----
```

STOP

Input for the program (Optional)

Output

===== Scenario 1 =====
Customer Added Successfully
Customer Details Updated
Customer Balance = 10000
===== Scenario 2 =====
Employee Hired Successfully
Employee Updated Successfully
Annual Salary = 600000
===== Scenario 3 =====
Account Opened Successfully
Account Closed Successfully
Total Balance = 25000

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35

```
SET SERVEROUTPUT ON;

-----
-- PACKAGE 2 : EmployeeManagement
-----

CREATE OR REPLACE PACKAGE EmployeeManagement AS
    PROCEDURE HireEmployee;
    PROCEDURE UpdateEmployee;
    FUNCTION CalculateAnnualSalary RETURN NUMBER;
END EmployeeManagement;
/

CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS

    PROCEDURE HireEmployee IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Employee Hired Successfully');
    END;

    PROCEDURE UpdateEmployee IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Employee Updated Successfully');
    END;

    FUNCTION CalculateAnnualSalary RETURN NUMBER IS
    BEGIN
        RETURN 600000;
    END;

END EmployeeManagement;
/

-----
-- PACKAGE 3 : AccountOperations
-----

CREATE OR REPLACE PACKAGE AccountOperations AS
    PROCEDURE OpenAccount;
    PROCEDURE CloseAccount;
    FUNCTION GetTotalBalance RETURN NUMBER;
END AccountOperations;
/

CREATE OR REPLACE PACKAGE BODY AccountOperations AS

    PROCEDURE OpenAccount IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Account Opened Successfully');
    END;

    PROCEDURE CloseAccount IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Account Closed Successfully');
    END;

    FUNCTION GetTotalBalance RETURN NUMBER IS
    BEGIN
        RETURN 25000;
    END;

END AccountOperations;
/

-----
-- CALLING ALL PACKAGES
-----
```

STOP

Input for the program (Optional)

Output

===== Scenario 1 =====
Customer Added Successfully
Customer Details Updated
Customer Balance = 10000
===== Scenario 2 =====
Employee Hired Successfully
Employee Updated Successfully
Annual Salary = 600000
===== Scenario 3 =====
Account Opened Successfully
Account Closed Successfully
Total Balance = 25000

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35

```
SET SERVEROUTPUT ON;

-----
-- PACKAGE 3 : AccountOperations
-----

CREATE OR REPLACE PACKAGE AccountOperations AS
    PROCEDURE OpenAccount;
    PROCEDURE CloseAccount;
    FUNCTION GetTotalBalance RETURN NUMBER;
END AccountOperations;
/

CREATE OR REPLACE PACKAGE BODY AccountOperations AS

    PROCEDURE OpenAccount IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Account Opened Successfully');
    END;

    PROCEDURE CloseAccount IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Account Closed Successfully');
    END;

    FUNCTION GetTotalBalance RETURN NUMBER IS
    BEGIN
        RETURN 25000;
    END;

END AccountOperations;
/

-----
-- CALLING ALL PACKAGES
-----
```

STOP

Input for the program (Optional)

Output

===== Scenario 1 =====
Customer Added Successfully
Customer Details Updated
Customer Balance = 10000
===== Scenario 2 =====
Employee Hired Successfully
Employee Updated Successfully
Annual Salary = 600000
===== Scenario 3 =====
Account Opened Successfully
Account Closed Successfully
Total Balance = 25000

Light 13 Wiki

```
-- CALLING ALL PACKAGES
```

```
BEGIN
```

```
DBMS_OUTPUT.PUT_LINE('==== Scenario 1 ====');
CustomerManagement.AddCustomer;
CustomerManagement.UpdateCustomer;
DBMS_OUTPUT.PUT_LINE('Customer Balance = ' ||
CustomerManagement.GetCustomerBalance);
```

```
DBMS_OUTPUT.PUT_LINE(' ');
```

```
DBMS_OUTPUT.PUT_LINE('==== Scenario 2 ====');
EmployeeManagement.HireEmployee;
EmployeeManagement.UpdateEmployee;
DBMS_OUTPUT.PUT_LINE('Annual Salary = ' ||
EmployeeManagement.CalculateAnnualSalary);
```

```
DBMS_OUTPUT.PUT_LINE(' ');
```

```
DBMS_OUTPUT.PUT_LINE('==== Scenario 3 ====');
AccountOperations.OpenAccount;
AccountOperations.CloseAccount;
DBMS_OUTPUT.PUT_LINE('Total Balance = ' ||
AccountOperations.GetTotalBalance);
```

```
END;
```

```
/
```

Output:

```
==== Scenario 1 ====
Customer Added Successfully
Customer Details Updated
Customer Balance = 10000
==== Scenario 2 ====
Employee Hired Successfully
Employee Updated Successfully
Annual Salary = 600000
==== Scenario 3 ====
Account Opened Successfully
Account Closed Successfully
Total Balance = 25000
```